

提取 Rhino 8 .3dm 文件预览图像的 Python 实现

1. Rhino 8 预览图像的存储方式

在 Rhino 8 的 .3dm 文件中，嵌入的预览缩略图被存储在文件的**属性 (Properties)** 部分，而且经过了压缩处理。与旧版本直接在文件中保存未压缩的位图不同，Rhino 6 及以上版本（包括 Rhino 8）对预览图像数据采用了 **zlib** 压缩，以减小文件体积¹。这意味着在 Rhino 8 的 .3dm 二进制中找不到标准 **BMP** (BM...) 或 **PNG** (\x89PNG) 文件头签名，因为图像数据并非明文存储，而是以压缩形式嵌入。

具体来说，Rhino 8 使用了 **OpenNURBS** 文件格式中的一个特殊区块来保存预览图像。该区块在 OpenNURBS 中被标识为 **TCODE_PROPERTIES_COMPRESSED_PREVIEWIMAGE**，表示“属性表中的压缩预览图像”²。它属于属性表 (Properties Table) 的子块，带有 CRC 校验标志。其完整的 32 位类型码为 **0x20008025** (十六进制)³。这个类型码包含了类别标识 (表记录 + CRC) 和具体的编号 **0x0025**。在较老的 Rhino 文件中，如果预览图未压缩，则会使用 **TCODE_PROPERTIES_PREVIEWIMAGE** (**0x20008023**) 来存储⁴。

简而言之，Rhino 8 的 .3dm 预览缩略图存储在属性表的一个压缩块中，不是加密的而是使用 **zlib** 压缩了原始位图数据¹。解压后得到的内容本质上是一个 **Windows DIB** (Device-Independent Bitmap) 图像数据，即包含 **BITMAPINFOHEADER** 和像素阵列的未压缩位图⁵。由于采用了压缩，无法直接通过文本扫描找到明文的 **BM** 或 **PNG** 头，这也解释了您在 Rhino 8 文件前几十 MB 中搜索不到惯用签名的原因。

参考：OpenNURBS 工具包通过 **zlib** 来压缩/解压模型中的网格和位图数据，包括预览图像在内¹。下面的论坛调试信息片段也印证了 Rhino 6+ 文件中预览图像块的存在和大小，其中 **TCODE_PROPERTIES_COMPRESSED_PREVIEWIMAGE** 出现，类型码 **20008025** (十六进制) 表示一个长度为 108,217 字节的压缩预览图像块³：

6

2. 提取预览图像的 Python 代码示例

无需依赖 Rhino 或 Windows Shell，您可以使用 Python 自行解析 .3dm 文件，定位并提取其中的预览缩略图。大体思路是直接读取二进制文件，找到表示预览图片的压缩数据块，解压缩得到位图数据，然后转换为常见图像格式。以下是一个可能的实现步骤：

- 读取文件：**以二进制模式打开 Rhino .3dm 文件，读取其内容到内存。由于文件可能较大，可酌情使用流式方法，但本例中为了简便直接读取整个文件。
- 定位缩略图块：**在数据中搜索预览图块的类型码标识。对 Rhino 8 文件，预览图块的类型码是 **0x20008025** (小端字节序表现为字节串 **\x25\x80\x00\x20**)。如果未找到该签名，可以尝试老版本未压缩缩略图的码 **0x20008023** (**\x23\x80\x00\x20**) 以防文件使用旧格式。
- 解析块长度：**一旦找到类型码位置，紧随其后的8个字节表示该块的数据长度 (OpenNURBS 在新版文件中 使用64位长度字段)⁶。读取这8字节并按小端格式解析出块长度。
- 提取块数据：**根据获得的长度，从文件中截取该块的完整数据段。其中末尾可能包含4字节的 CRC 校验值 (因为类型码包含 **TCODE_CRC** 标志)。如果长度包含了CRC，这4字节需要在解压前剔除。
- 解压缩数据：**根据块类型判断是否压缩。如果类型码是 **0x20008025** (压缩预览)，则使用 **zlib.decompress** 来解压获得原始图像字节数据¹；如果类型码是 **0x20008023** (未压缩预览)，则直接使用其数据 (去除CRC后)。

6. **转换图像格式**：解压得到的 `image_data` 即为 Windows DIB 格式的位图数据⁵。您可以将其转换成常规图像格式以供前端使用。例如，可以在其前面加上 BMP 文件头并保存为 `.bmp` 文件，或借助 Pillow (PIL) 将其转为 PNG/JPG 格式。下面提供一个将 DIB 数据存为 BMP 并输出的代码片段。

```
import struct, zlib

# 读取 Rhino .3dm 文件的二进制内容
with open('input.3dm', 'rb') as f:
    data = f.read()

# 寻找预览图像块的类型码 (小端序列)
sig_compressed = b'\x25\x80\x00\x20' # TCODE_PROPERTIES_COMPRESSED_PREVIEWIMAGE
sig_uncompressed = b'\x23\x80\x00\x20' # TCODE_PROPERTIES_PREVIEWIMAGE (旧格式)
idx = data.find(sig_compressed)
sig = sig_compressed
if idx == -1:
    # 如果未找到压缩块，则尝试未压缩块标识
    idx = data.find(sig_uncompressed)
    sig = sig_uncompressed

if idx != -1:
    # 解析8字节长度 (小端64位整数)
    length = struct.unpack('<Q', data[idx+4: idx+12])[0]
    chunk_data = data[idx+12 : idx+12 + length]
    # 如有CRC，长度字段通常包含CRC在内，因此实际图像数据需去掉最后4字节CRC校验
    # 判断块类型并处理解压
    if sig == sig_compressed:
        compressed_bytes = chunk_data[:-4] # 去除CRC
        image_data = zlib.decompress(compressed_bytes) # 解压出位图数据
    else:
        image_data = chunk_data[:-4] # 未压缩，直接取数据去除CRC

    # 此时 image_data 是BITMAPINFOHEADER+像素数据的DIB字节流
    # 构造 BMP 文件头以保存成标准图像 (可选，也可以用PIL处理)
    dib = image_data
    header_size = struct.unpack('<I', dib[0:4])[0] # BITMAPINFOHEADER 大小，一般为40字节
    # 计算 BMP 文件头信息
    file_size = 14 + len(dib) # BMP文件总大小 = 文件头14字节 + DIB数据大小
    pixel_offset = 14 + header_size # 像素数据在文件中的起始偏移 = 14 + DIB头长度 (假定无调色板)
    bmp_header = b'BM' + struct.pack('<I', file_size) + b'\x00\x00\x00\x00' + struct.pack('<I', pixel_offset)
    bmp_bytes = bmp_header + dib
    # 将BMP字节流写出到文件
    with open('preview.bmp', 'wb') as img_file:
        img_file.write(bmp_bytes)
    print("预览图像已提取并保存为 preview.bmp")
```

```
else:
    print("未找到预览图像数据块，请确认文件版本或完整性。")
```

上述代码会在当前目录生成一个 `preview.bmp`，其中包含提取出的缩略图。我们通过搜索特定的类型码来定位预览图所在的位置，然后读取长度、解压得到原始位图字节流，最后加上标准 BMP 文件头保存。从代码可以看出，Rhino 8 的预览图像数据需要使用 `zlib.decompress()` 来解码 ¹。

说明：如果需要得到 PNG/JPG 等格式以便网页前端展示，可以在获取到 `image_data`（DIB 数据）后，使用 Python 的 Pillow 库进行转换。例如：

```
from PIL import Image
from io import BytesIO
img = Image.open(BytesIO(bmp_bytes))
img.save('preview.png')
```

这样即可将提取的缩略图转换为 PNG 格式。同时请注意，如果 `bit_count <= 8`（位图是 256 色或更少），还需要考虑调色板大小；但通常 Rhino 预览图是 24 位或 32 位真彩色，无调色板问题。

3. 预览图像块的类型码与解压方法

综上所述，Rhino 8 预览缩略图所在块的类型码（TCODE）为 `0x20008025`（十六进制）³。这一数值对应于 OpenNURBS 定义的 `TCODE_PROPERTIES_COMPRESSED_PREVIEWIMAGE`，表示该块数据经过了压缩处理。如果查阅 OpenNURBS 常量，会发现还有一个 `TCODE_BITMAPPREVIEW`（`TCODE_INTERFACE | 0x0014`）的定义，但那指的是旧的界面相关标识 ⁷，实际文件中存储预览图用的是属性表下的上述代码。

数据解压：预览块的数据采用 `zlib` (Deflate) 压缩算法存储 ¹。要恢复出图像，需要按照前述步骤截取出该块的压缩字节流，然后使用 `zlib.decompress()` 解压得到原始位图数据。解压后您将获得标准的 DIB/BMP 位图内容 ⁵。通过添加 BMP 文件头或利用图像库，即可将其还原为常见的图像文件查看。例如，上面的代码片段演示了如何利用 Python 自带的 `zlib` 模块完成解压；我们引用的 OpenNURBS 文档也明确说明，OpenNURBS 工具包正是使用 `zlib` 来压缩和解压包括预览图在内的位图数据 ¹。

最后，验证 CRC 校验值是可选的步骤。OpenNURBS 在压缩块末尾附加了 32 位 CRC 校验码，以确保数据完整性 ³。在读取时，您可以通过计算解压前数据的 CRC32 来比对校验码是否一致。但在大多数情况下，如果解压成功且没有异常，CRC 验证不是必需的。总而言之，按照上述方法提取并解压缩，就能够成功获得 Rhino 8 `.3dm` 文件中的嵌入预览图像。

参考资料：

- Rhino C++ 开发文档：“Rhino 将文档的缩略图预览图像存储为 `ON_WindowsBitmap`，本质上是未压缩的 Windows DIB（设备无关位图）” ⁵
- OpenNURBS 源码/文档：预览图像块类型码定义及压缩标志 ² ³
- OpenNURBS 工具包说明：“openNURBS 工具包使用 `zlib` 来压缩和解压网格以及位图（包含预览图像）” ¹
- McNeel 论坛讨论：Rhino 6 文件结构调试输出，显示 `TCODE_PROPERTIES_COMPRESSED_PREVIEWIMAGE` 块及其长度 ⁶（证明预览图像以压缩形式存储）。

1 **GitHub - hansec/opennurbs: Open CAD library for NURBS representations**

<https://github.com/hansec/opennurbs>

2 4 7 **Rhino C++ API: OpenNURBS**

https://developer.rhino3d.com/api/cpp/group__open_n_u_r_b_s.html

3 6 **Help with recovering 3dm file - Rhino - McNeel Forum**

<https://discourse.mcneel.com/t/help-with-recovering-3dm-file/203749>

5 **Rhino - Extracting Thumbnail Preview Images**

<https://developer.rhino3d.com/guides/cpp/extracting-thumbnail-preview-images/>